GPUs: From Graphics to AI

Part 1: Why a GPU Is Not a CPU

Hardware Description Languages

Three Sessions

Part 1 CPU versus GPU, and how the GPU evolved

Part 2 Programming the GPU with CUDA C++

and the libraries built on top of it

Part 3 The GPU as an AI architecture

MAC units and systolic arrays in SystemVerilog

Why Should an HDL Class Care?

GPUs are written in an HDL, just like your labs

They are among the largest chips ever built

over **200 billion** transistors

Nobody draws 200 billion transistors by hand

The trick is **one small module, replicated**

Two Ways to Be Fast

Latency: how long does one task take?

measured in seconds per task

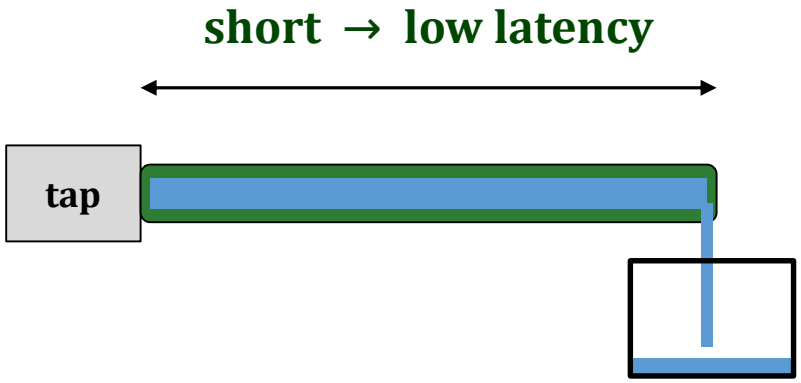
Throughput: how many tasks finish per second?

measured in tasks per second

A CPU is built for latency

A GPU is built for throughput

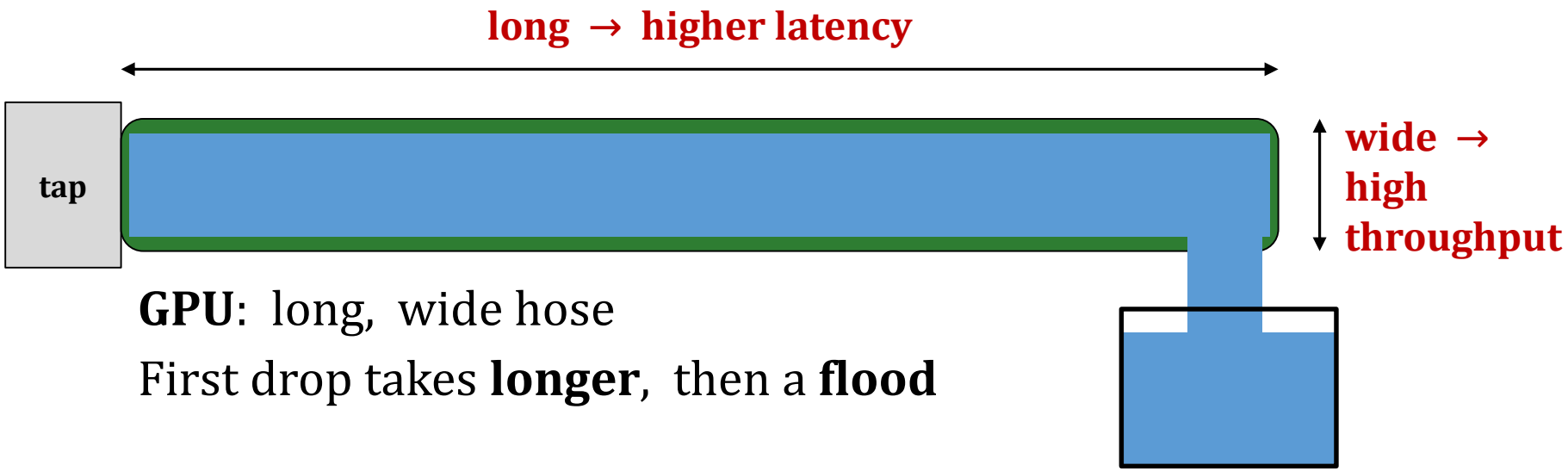
Latency and Throughput: A Hose



CPU: short, narrow hose

First drop arrives **quickly**

Then only a **trickle**



GPU: long, wide hose

First drop takes **longer**, then a **flood**

Latency = time until the first drop comes out (hose length)

Throughput = litres per second once it flows (hose width)

Car or Bus?

Sports car
2 people

Trip: **10 minutes**

Moves **12 people / hour**

Bus
60 people

Trip: **30 minutes**

Moves **120 people / hour**

low latency = CPU

high throughput = GPU

Neither is faster. They are fast at different things.

Where a CPU Spends Transistors

A few **large, smart** cores

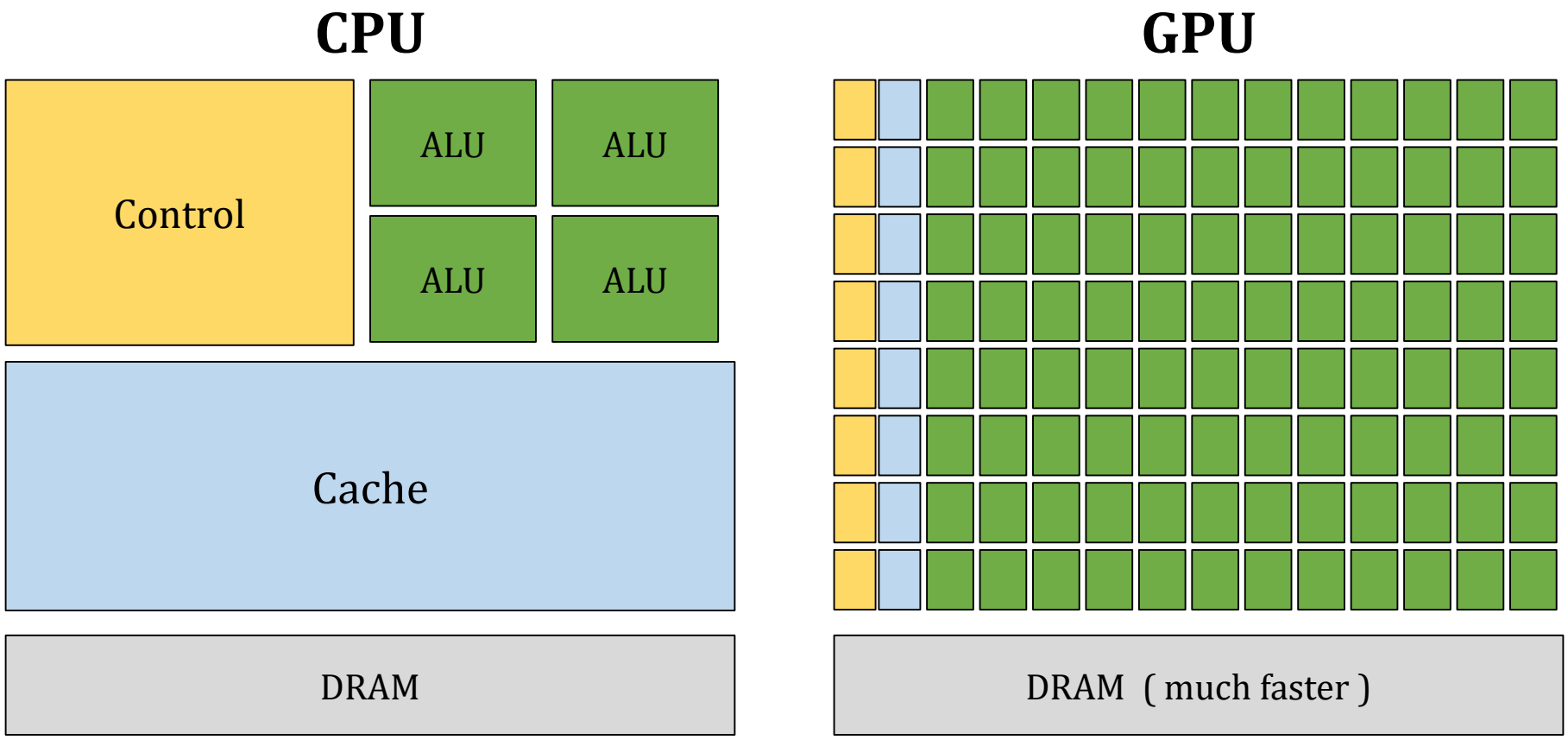
Big caches so data is usually close

Branch prediction: guess which way an if goes

Out-of-order execution: run whatever is ready

All of it exists to make **one thread** finish sooner

Same Silicon, Different Priorities



Amber = control Blue = cache Green = arithmetic

Where a GPU Spends Transistors

Thousands of **small, simple** arithmetic units

Little control per unit:

many units share **one** instruction decoder

Small caches per unit

A very wide path to memory

Memory Is Slow: The CPU Answer

A trip to DRAM costs hundreds of clock cycles

CPU strategy: **avoid the wait**

- deep cache hierarchy L1, L2, L3
- prefetch what you will probably need
- speculate past the stall

Works well when the working set fits in cache

Memory Is Slow: The GPU Answer

GPU strategy: **tolerate the wait**

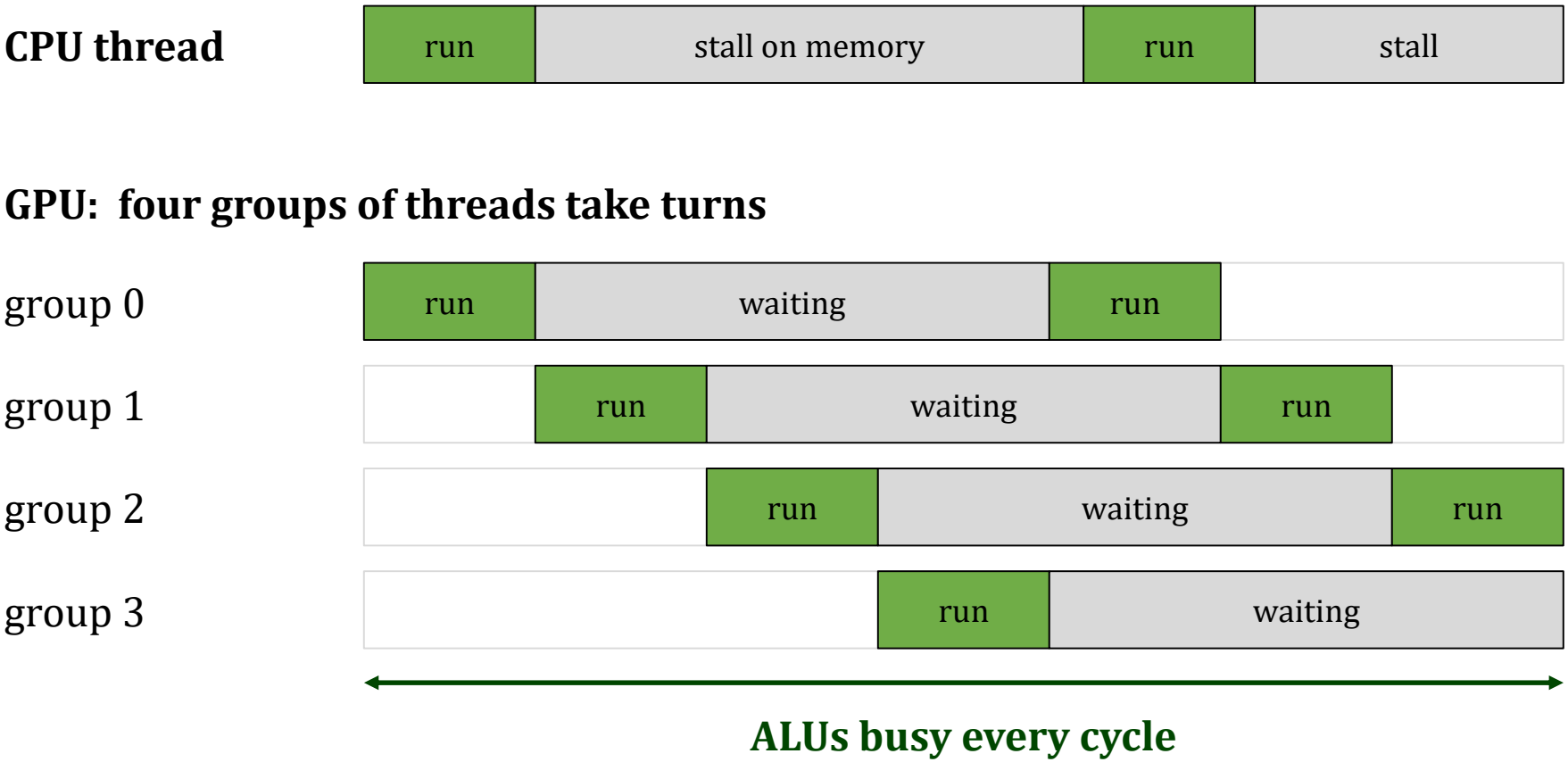
Keep many groups of threads resident at once

When one group stalls on memory,

switch to another group - in one clock

Every thread is slow; the machine is busy

Hiding Latency With Threads



Each group still waits. The **hardware** never does.

Flynn's Taxonomy

SISD single instruction, single data

a simple CPU core

SIMD single instruction, multiple data

vector units: SSE, AVX, NEON

MISD multiple instruction, single data

a fault tolerant CPU

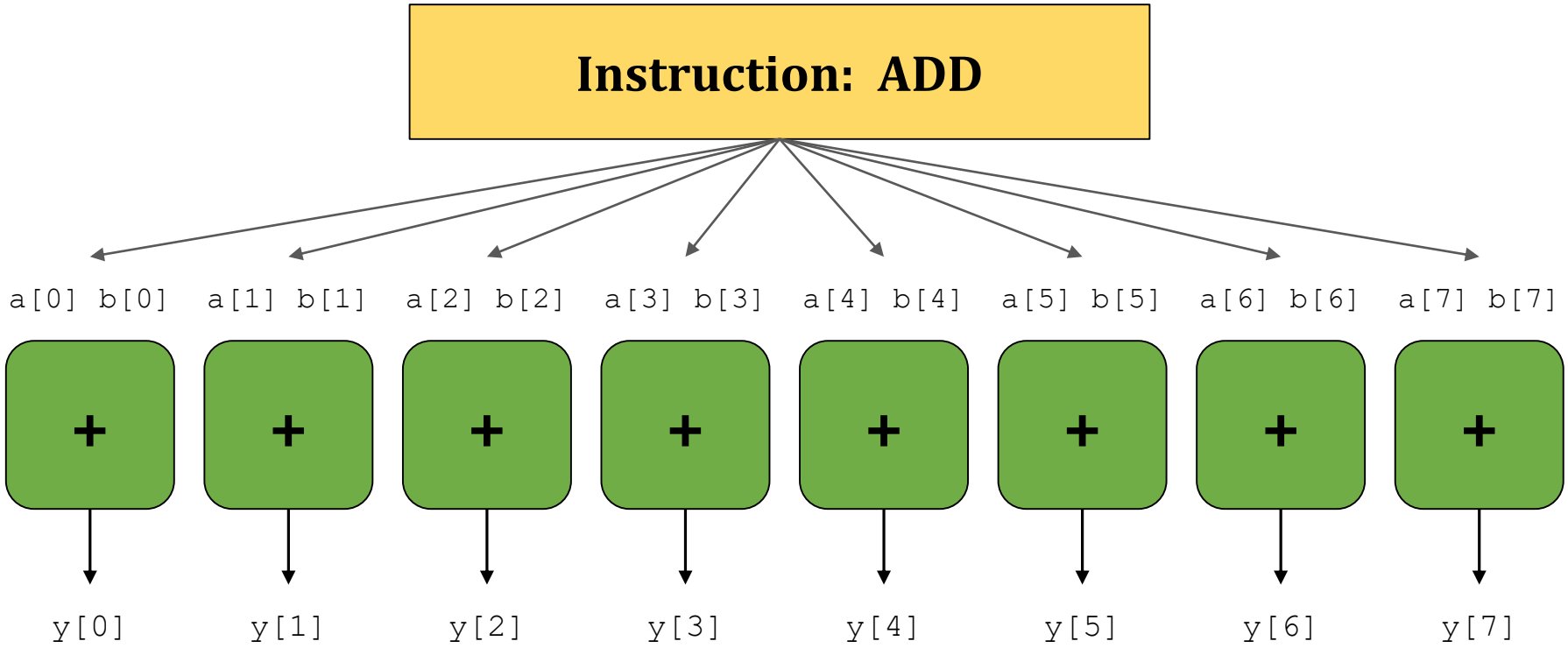
MIMD multiple instruction, multiple data

a multicore CPU

SIMT single instruction, multiple **threads**

the GPU (NVIDIA's term)

SIMD: One Decoder, Many Lanes



Fetch and decode **once**. Do the work **eight** times.

SIMD in SystemVerilog

```
module simd_add #(
    parameter int LANES = 8,
    parameter int W     = 16
) (
    input  logic                 clk, valid_in,
    input  logic [LANES*W-1:0] a, b,
    output logic [LANES*W-1:0] y,
    output logic                 valid_out
);
    // control:  built once
    always_ff @(posedge clk) valid_out <= valid_in;

    // datapath:  replicated LANES times
    for (genvar i = 0; i < LANES; i++) begin : g_lane
        always_ff @(posedge clk)
            if (valid_in)
                y[i*W +: W] <= a[i*W +: W] + b[i*W +: W];
    end
endmodule
```

Part Select in SystemVerilog

```
logic [15:0] data;
logic [7:0]  lower_byte;

// Selects 8 bits starting from index 8
//                 and going upwards (8 to 15)
assign lower_byte = data[8 +: 8];

// Equivalent to data[15:8] depending on
//        declaration, or data[8:15]
```

Part Select in SystemVerilog

```
module bus_splitter (
   input  logic [31:0]       wide_bus,
   output logic [7:0][3:0]   split_bytes // An array of 4 bytes
);
   always_comb begin
      // Loop through each of the 4 byte positions
      for (int i = 0; i < 4; i++) begin
         // base  = i * 8 (0, 8, 16, 24)
         // width = 8 bits fixed
         split_bytes[i] = wide_bus[(i * 8) +: 8];
      end
   end
endmodule
```

SIMD in SystemVerilog

```
module simd_add #(
    parameter int LANES = 8,
    parameter int W     = 16
) (
    input  logic                 clk, valid_in,
    input  logic [LANES*W-1:0] a, b,
    output logic [LANES*W-1:0] y,
    output logic                 valid_out
);
    // control:  built once
    always_ff @(posedge clk) valid_out <= valid_in;

    // datapath:  replicated LANES times
    for (genvar i = 0; i < LANES; i++) begin : g_lane
        always_ff @(posedge clk)
            if (valid_in)
                y[i*W +: W] <= a[i*W +: W] + b[i*W +: W];
    end
endmodule
```

What the Code Tells Us

Control cost is **fixed**

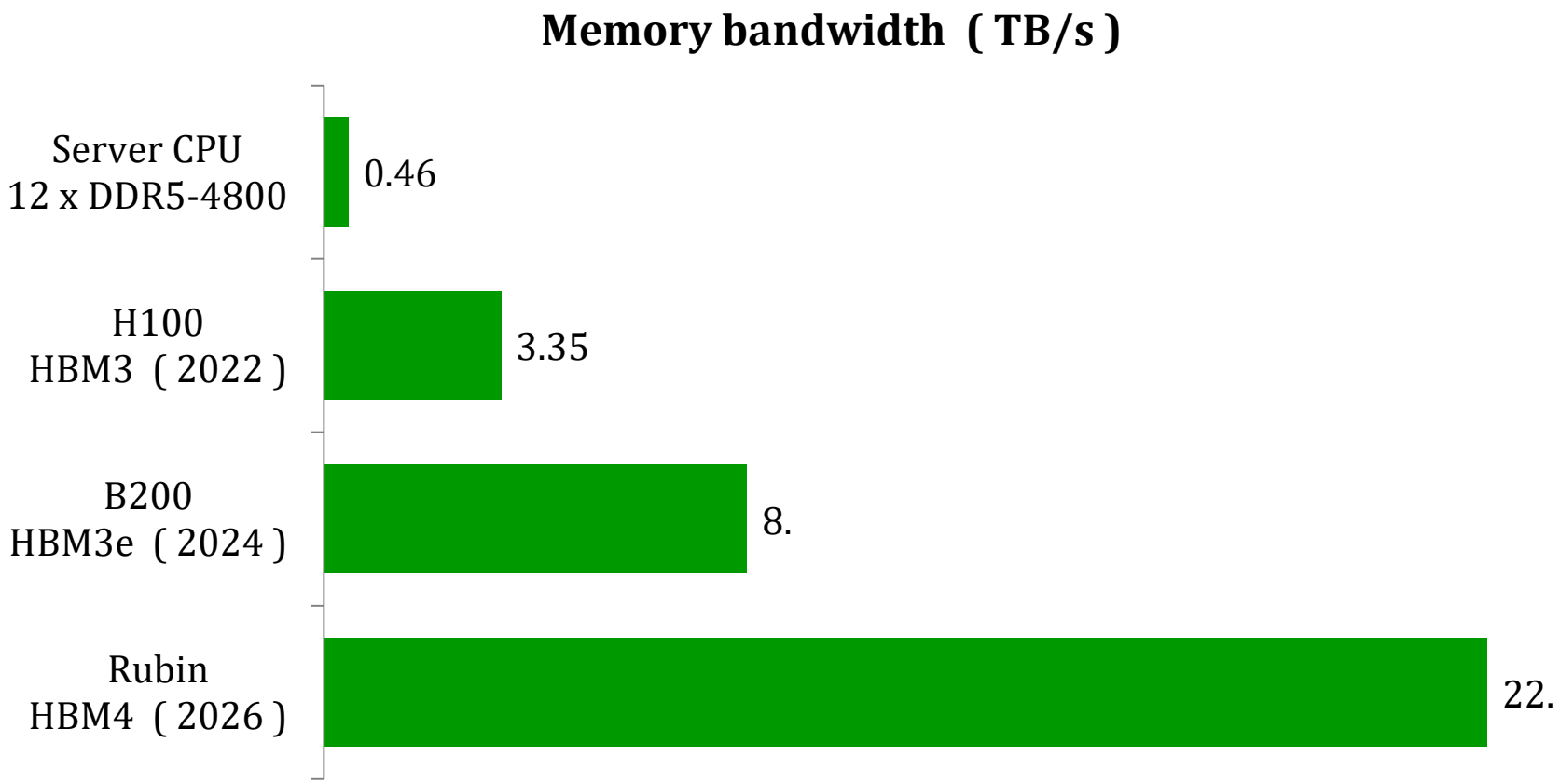
Datapath cost grows with **LANES**

Double the lanes:

- almost double the arithmetic
- for much less than double the area

This is how a GPU scales to thousands of ALUs

Feeding the ALUs



Thousands of ALUs are useless if data cannot reach them

Which One Should Do the Work?

CPU wins

Long serial chains

Lots of if / else

Small data sets

Pointer chasing:

linked lists, trees

The operating system

GPU wins

Same operation on

millions of elements

Pixels and vertices

Matrix multiply

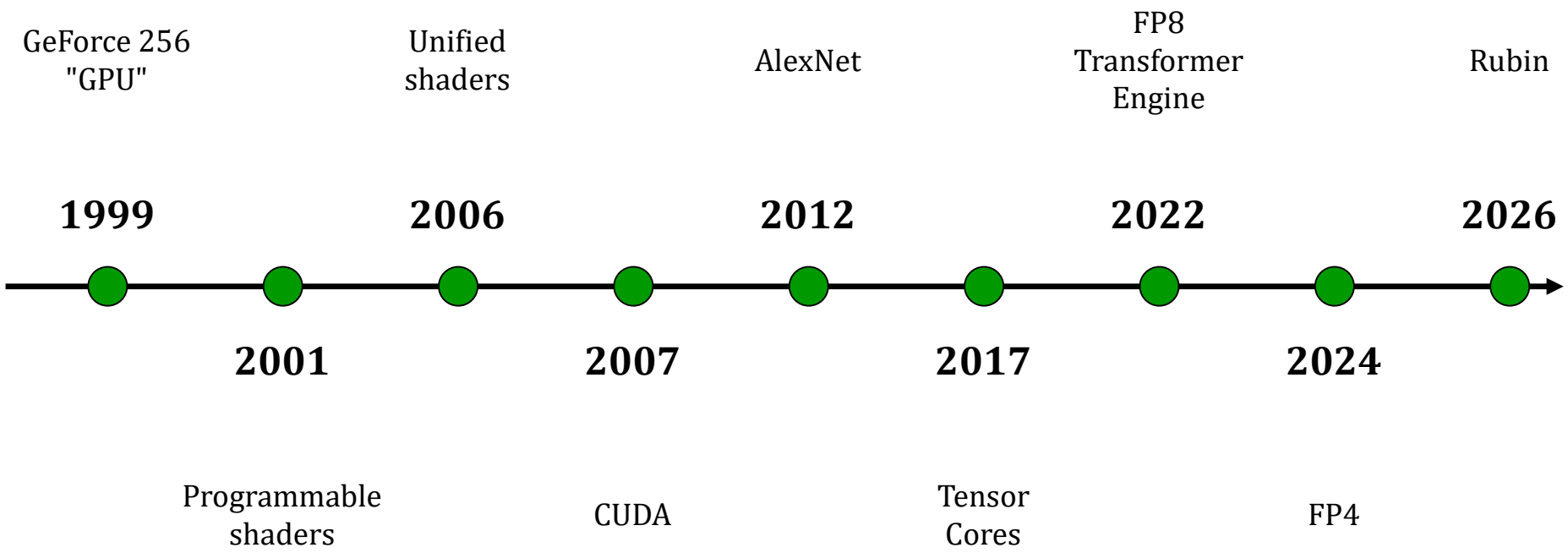
Neural networks

Physics, signal processing

How the GPU Evolved

from fixed-function graphics to AI accelerator

Twenty-Seven Years in One Line



Graphics → general purpose → AI

Before the GPU

1990s 3D graphics cards were **fixed-function**

Hard-wired circuits for each step of drawing

texture a triangle, blend pixels, test depth

Fast, but they could do **only** what was wired in

Essentially a big state machine, not a processor

The Graphics Pipeline



Millions of vertices, millions of pixels

Each one handled **independently** of its neighbours

A perfect fit for many simple parallel units

1999: The First "GPU"

NVIDIA **GeForce 256**

Marketed as the first **G**raphics **P**rocessing **U**nit

Moved **transform and lighting** from the CPU

onto the graphics chip

Still fixed-function, but now the whole pipeline

lived on one chip

2001: Programmable Shaders

GeForce 3 and DirectX 8

A **shader** is a small program

run once per vertex, or once per pixel

Now the stages have an instruction set

The same program runs on every pixel:

that is **SIMD**

The GPGPU Hack

Early 2000s: researchers noticed the GPU

was much faster than the CPU at arithmetic

So they **disguised** math as graphics

arrays become textures

loop bodies become pixel shaders

results are read back as an image

It worked, but it was painful

2006: Unified Shaders

NVIDIA **G80** (GeForce 8800)

Before: separate vertex units and pixel units

one kind often sat idle

After: **one** kind of core does every job

128 identical streaming processors

A pool of identical cores is a general computer

2007: CUDA

Compute **U**nified **D**evice **A**rchitecture

Program the GPU in C, with no graphics at all

Write a function, launch it on thousands of threads

Integers, pointers, arbitrary reads and writes

Session 2 is entirely about this

2012: AlexNet

A deep neural network for image recognition

Trained on **two** gaming GPUs (GTX 580)

Won ImageNet 2012 by a wide margin

The lesson for the field:

deep learning works when you have enough compute and GPUs supply it cheaply

Why Neural Networks Fit GPUs

A network layer is a **matrix multiply**

A matrix multiply is millions of

multiply-accumulate (MAC) operations

acc <= acc + a × b

Every MAC in a layer is independent

The same workload shape as shading pixels

The GPU Starts Changing for AI

2016 Pascal P100

HBM2 memory stacked beside the die

NVLink between GPUs

fast 16-bit floating point

2017 Volta V100

Tensor Cores: hardware for small matrix multiplies

Smaller Numbers, More Math

2020 Ampere A100

TF32 and BF16 formats, structured sparsity

- **TF32**: A proprietary 19-bit math mode and internal format introduced by NVIDIA for Tensor
- **Bit Breakdown**: Uses 1 sign bit, 8 exponent bits (matching standard FP32 range), and 10 mantissa/fraction bits (matching standard FP16 precision).
- **Purpose**: Acts as a drop-in replacement for standard 32-bit single-precision floating-point (FP32). It allows code written for FP32 to run much faster on Tensor Cores without any modifications, providing FP32's dynamic range with FP16's computational speed.

- **BF32n**: A 16-bit floating-point format originally designed by Google Brain for machine learning.
- **Bit Breakdown**: Uses 1 sign bit, 8 exponent bits (matching FP32 range), and 7 mantissa/fraction bits (truncated precision).
- **Purpose**: Replaces traditional 16-bit half-precision (`FP16`) in many deep learning tasks. Traditional FP16 has a narrow range and frequently overflows or underflows, requiring complex scaling techniques. BF16 keeps the same large number range as FP32, preventing overflow errors while cutting memory usage in half.

Smaller Numbers, More Math

2022 Hopper H100

Transformer Engine picks precision per layer

80 billion transistors

8-bit floating point (FP8)

8-bit floating point (FP8)

- **Purpose:** It cuts the memory footprint of large language models (LLMs) and deep learning systems in half compared to 16-bit formats (like FP16 or BF16), speeding up AI training and inference.
- **E4M3:** Uses 4 exponent bits and 3 mantissa bits (plus 1 sign bit). It offers higher precision and is ideal for forward-pass weights and activations.
- **E5M2:** Uses 5 exponent bits and 2 mantissa bits (plus 1 sign bit). It trades precision for a much wider dynamic range, making it better suited for handling large gradient spikes during model training.

Today's AI GPUs

2024 Blackwell B200

two dies acting as one GPU, 208 billion transistors

4-bit floating point (FP4)

192 GB HBM3e at 8 TB/s

2026 Rubin

336 billion transistors, two compute dies

288 GB HBM4 at 22 TB/s

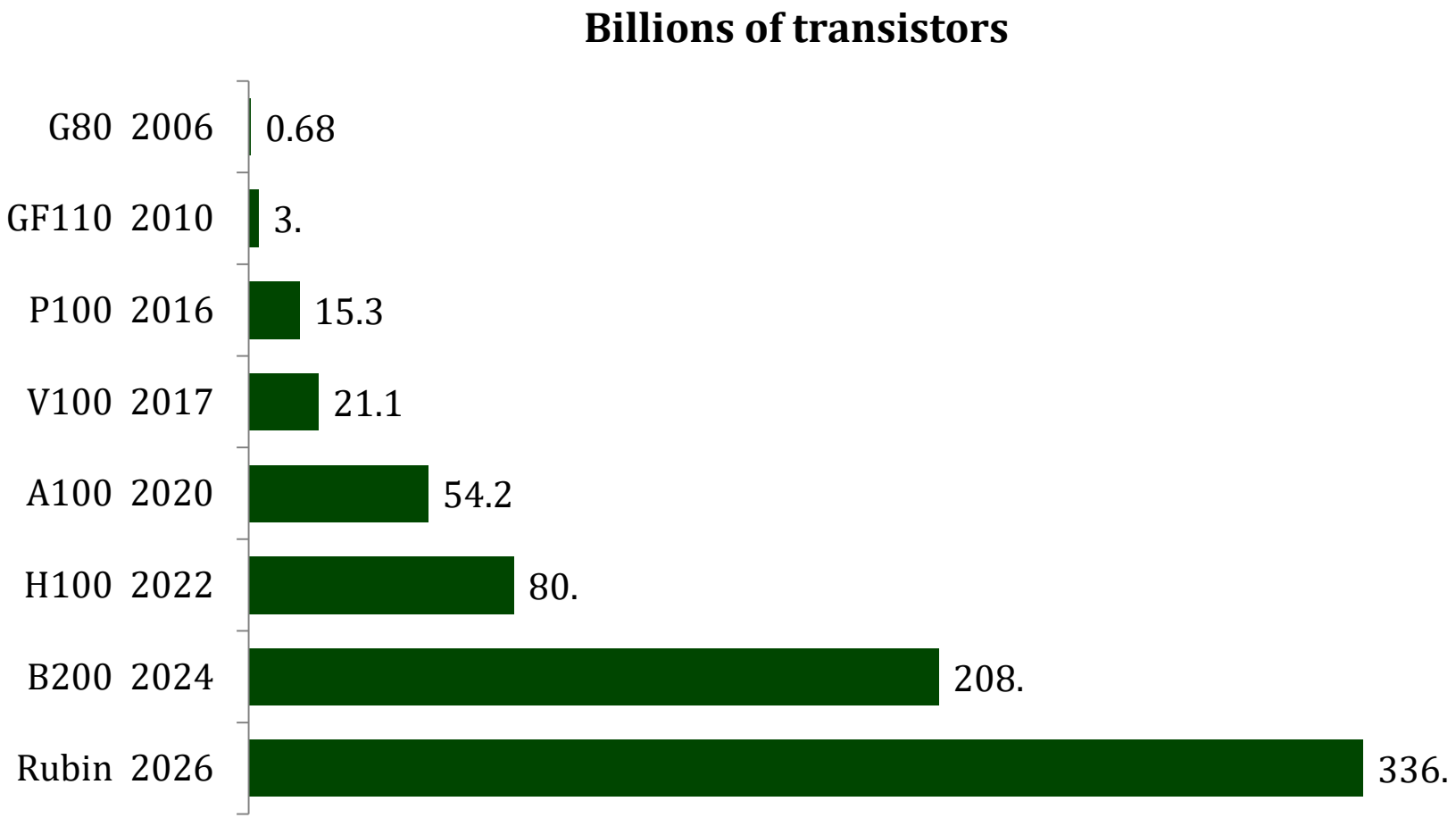
4-bit floating point (FP4)

Bit Layout: A typical FP4 format uses 4 total bits divided into 1 sign bit, 2 exponent bits, and 1 mantissa (fraction) bit (known as E2M1).

Two-Level Scaling: Because 4 bits have very little precision on their own, formats like NVIDIA's **NVFP4** use block scaling. They group 16 values together with a local FP8 scale, plus a global FP32 scale for the whole tensor.

Mixed Precision: Computation happens in fast FP4, but results accumulate in higher precision formats like FP16 or FP32 to prevent major math errors.

Transistors per GPU



About 500 times more in twenty years

What Changed, What Stayed

Stayed the same

many simple units, one instruction for many

hide memory latency with parallel threads

Changed

graphics stages → general cores

general cores → matrix engines

more bandwidth, smaller number formats

Key Ideas from Part 1

CPU: minimize **latency** of one thread

GPU: maximize **throughput** of many threads

SIMD: pay for control once, replicate the datapath

GPUs became AI chips because neural networks

are mostly multiply-accumulate

Let's Build a SIMD Unit

Exercise: Parameterized SIMD Adder

1. Type in `simd_add` and write a testbench
   that drives random a, b and checks every lane
2. Synthesize for LANES = 4, 8, 16, 32
   record cell count or LUTs for each
3. Plot area against LANES. What fraction is control?
4. Add a sub input that selects a + b or a − b
   for **all** lanes at once. Where does the mux go?